

Nginx 日志管理与分析

一、学习目标

- 理解 Nginx 访问日志和错误日志的作用
- 掌握日志格式自定义配置方法
- 学会日志切割与轮转管理
- 掌握常用日志分析命令和技巧
- 了解日志安全与合规要求

二、理论知识

2.1 Nginx 日志概述

日志是服务器的"黑匣子", 记录着服务器运行的每一个细节。对于运维人员来说, 日志是排查问题、分析性能、统计访问量的重要依据。

Nginx 有两种主要日志类型: 访问日志 (Access Log) 和错误日志 (Error Log)。

访问日志 (Access Log)

访问日志记录每一次客户端请求的详细信息, 包括访问时间、客户端 IP、请求的 URL、响应状态码、响应大小、来源页面、用户代理等。这些信息可以用于流量统计、用户行为分析、安全审计等。

就像商店的门禁记录, 每位顾客进店的时间、停留多久、买了什么都会被记录下来。

错误日志 (Error Log)

错误日志记录 Nginx 运行过程中遇到的各种问题, 包括配置错误、连接失败、上游服务器故障等。错误日志是排查问题的第一手资料。

就像机器的故障报警器, 设备出现任何异常都会发出警报并记录下来。

2.2 访问日志格式详解

Nginx 访问日志的格式通过 `log_format` 指令定义，默认使用名为 `main` 或 `combined` 的格式。

默认日志格式：

```
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" "$http_x_forwarded_for"';
```

日志字段含义：

变量	含义	示例
\$remote_addr	客户端 IP 地址	192.168.1.100
\$remote_user	认证用户名	- （未认证时为空）
\$time_local	本地时间	30/Dec/2025:10:30:00 +0800
\$request	请求行	GET /index.html HTTP/1.1
\$status	响应状态码	200
\$body_bytes_sent	发送的响应体大小	1234
\$http_referer	来源页面	https://www.google.com/
\$http_user_agent	用户代理（浏览器信息）	Mozilla/5.0...
\$http_x_forwarded_for	代理链中的原始 IP	10.0.0.1

实际日志示例：

```
192.168.1.100 - - [30/Dec/2025:10:30:00 +0800] "GET /index.html
HTTP/1.1" 200 1234 "https://www.google.com/" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36"
```

2.3 错误日志级别

错误日志有多个级别，从低到高分别是：debug、info、notice、warn、error、crit、alert、emerg。级别越高表示问题越严重。

级别	含义	使用场景
debug	调试信息	开发调试时使用，信息最详细
info	一般信息	记录正常运行信息
notice	通知	正常但值得注意的情况
warn	警告	可能有问题但不影响运行
error	错误	发生错误但服务仍可运行
crit	严重	严重问题需要立即处理
alert	警报	必须立即采取行动
emerg	紧急	系统不可用

生产环境通常使用 `warn` 或 `error` 级别，避免日志量过大；调试时可以临时开启 `debug` 级别获取详细信息。

2.4 日志切割的必要性

随着网站运行，日志文件会持续增长。如果不进行管理，单个日志文件可能达到几十 GB 甚至更大，这会导致以下问题：

磁盘空间耗尽： 日志不断增长最终会占满磁盘空间，导致服务器无法正常运行。

查看分析困难： 打开或搜索超大文件非常耗时，影响问题排查效率。

备份困难： 大文件的备份和传输都很困难。

日志切割策略：

按时间切割是最常用的方式，比如每天、每周生成一个新的日志文件。按大小切割适用于访问量不稳定的场景，当文件达到一定大小时自动切割。混合策略则结合时间和大小两个条件。

2.5 日志分析的价值

日志分析可以帮助我们：

流量监控： 了解网站的访问量、高峰时段、访问趋势。

性能优化： 找出响应慢的请求、高频访问的资源。

安全审计： 发现异常访问、攻击行为、非法请求。

用户分析： 了解用户来源、使用的设备、访问路径。

故障排查： 通过错误日志快速定位问题原因。

常用的分析维度包括：PV（页面访问量）、UV（独立访客数）、状态码分布、热门页面、访问来源、地理分布等。

2.6 日志安全与合规

安全注意事项：

日志可能包含敏感信息如用户 IP、访问行为等，需要妥善保管。应设置适当的文件权限，只允许必要的用户访问。敏感环境下可能需要对日志进行脱敏处理。

合规要求：

根据相关法规（如 GDPR、网络安全法），日志需要保存一定期限。同时也需要有日志删除机制，不能无限期保留用户数据。建议根据业务需要和法规要求制定日志保留策略，通常保留 30 天到 1 年不等。

三、实践内容

3.1 实验环境准备

实验目的：掌握 Nginx 日志配置、切割和分析技巧

前置条件：已完成前三章实验，Nginx 已正常运行

3.2 实验一：查看默认日志配置

操作步骤：

步骤1：查看当前日志配置

```
grep -E "access_log|error_log|log_format" /etc/nginx/nginx.conf
```

步骤2：查看默认访问日志位置

```
ls -la /var/log/nginx/
```

步骤3：查看访问日志内容

```
sudo tail -20 /var/log/nginx/access.log
```

步骤4：查看错误日志内容

```
sudo tail -20 /var/log/nginx/error.log
```

步骤5：实时监控访问日志

```
sudo tail -f /var/log/nginx/access.log
```

（在另一个终端访问网站产生日志，然后 Ctrl+C 停止监控）

步骤6：查看日志文件大小

```
du -sh /var/log/nginx/*
```

3.3 实验二：自定义日志格式

操作步骤：

```
# 步骤1: 备份原配置
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak3

# 步骤2: 编辑 nginx.conf 添加自定义日志格式
sudo tee /etc/nginx/nginx.conf << 'EOF'
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log warn;
pid /run/nginx.pid;

include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
    use epoll;
    multi_accept on;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # ===== 日志格式定义 =====

    # 默认格式 (标准 combined 格式)
    log_format main '$remote_addr - $remote_user [$time_local]
"$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';

    # 详细格式 (增加响应时间、连接信息)
    log_format detailed '$remote_addr - $remote_user [$time_local]
"$request" '
                       '$status $body_bytes_sent "$http_referer" '
                       '"$http_user_agent" '
                       'rt=$request_time uct=$upstream_connect_time
'
```

```

        'uht=$upstream_header_time
    urt=$upstream_response_time';

# JSON 格式（便于日志分析工具处理）
log_format json_log escape=json '{'
    '"time": "$time_iso8601",'
    '"remote_addr": "$remote_addr",'
    '"remote_user": "$remote_user",'
    '"request": "$request",'
    '"status": $status,'
    '"body_bytes_sent": $body_bytes_sent,'
    '"referer": "$http_referer",'
    '"user_agent": "$http_user_agent",'
    '"request_time": $request_time,'
    '"upstream_response_time": "$upstream_response_time"'
    '}'

# 简洁格式（节省空间）
log_format minimal '$remote_addr [$time_local] "$request" $status
$body_bytes_sent';

# ===== 日志格式定义结束 =====

# 默认访问日志
access_log /var/log/nginx/access.log main;

# 性能优化
sendfile on;
tcp_nopush on;
tcp_nodelay on;
keepalive_timeout 65;
types_hash_max_size 4096;

# 引入站点配置
include /etc/nginx/conf.d/*.conf;
}
EOF

# 步骤3：测试配置
sudo nginx -t

# 步骤4：重新加载

```

```
sudo systemctl reload nginx
```

日志格式变量说明：

\$request_time	请求处理总时间（秒）
\$upstream_connect_time	与上游服务器建立连接的时间
\$upstream_header_time	收到上游响应头的时间
\$upstream_response_time	收到上游完整响应的时间
\$time_iso8601	ISO 8601 格式时间
escape=json	对特殊字符进行 JSON 转义

3.4 实验三：为不同站点配置独立日志

操作步骤：

```
# 步骤1：创建日志目录
sudo mkdir -p /var/log/nginx/sites

# 步骤2：修改站点配置使用独立日志
sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'
server {
    listen 8082;
    server_name localhost;

    root /var/www/static-site;
    index index.html;
    charset utf-8;

    # 使用独立的日志文件和详细格式
    access_log /var/log/nginx/sites/static-site_access.log detailed;
    error_log /var/log/nginx/sites/static-site_error.log warn;

    # Gzip 压缩
    gzip on;
    gzip_min_length 1024;
    gzip_comp_level 5;
    gzip_types text/plain text/css text/xml text/javascript
```



```

        application/json application/javascript
application/xml
        image/svg+xml;
gzip_vary on;

location / {
    try_files $uri $uri/ =404;
}

location ~* \.(css|js)$ {
    expires 7d;
    add_header Cache-Control "public, immutable";
}

location ~* \.(jpg|jpeg|png|gif|ico|svg|webp)$ {
    expires 30d;
    access_log off; # 图片请求不记录日志
}
}
EOF

```

步骤3: 创建 JSON 日志格式站点 (用于日志分析演示)

```

sudo tee /etc/nginx/conf.d/json-log-site.conf << 'EOF'
server {
    listen 8083;
    server_name localhost;

    root /var/www/static-site;
    index index.html;

    # 使用 JSON 格式日志
    access_log /var/log/nginx/sites/json_access.log json_log;
    error_log /var/log/nginx/sites/json_error.log warn;

    location / {
        try_files $uri $uri/ =404;
    }
}
EOF

```

步骤4: 测试并重载

```

sudo nginx -t

```

```
sudo systemctl reload nginx

# 步骤5: 开放端口
sudo firewall-cmd --permanent --add-port=8083/tcp
sudo firewall-cmd --reload

# 步骤6: 产生一些访问日志
for i in {1..10}; do
    curl -s http://localhost:8082/ > /dev/null
    curl -s http://localhost:8083/ > /dev/null
    curl -s http://localhost:8082/css/style.css > /dev/null
    curl -s http://localhost:8082/nonexistent > /dev/null
done

# 步骤7: 查看不同格式的日志
echo "=== 详细格式日志 ==="
sudo tail -3 /var/log/nginx/sites/static-site_access.log

echo ""
echo "=== JSON 格式日志 ==="
sudo tail -3 /var/log/nginx/sites/json_access.log
```

3.6 实验五：配置日志缓冲

高流量网站频繁写入日志会影响性能，可以使用日志缓冲。

操作步骤：

```
# 步骤1: 配置带缓冲的日志
sudo tee /etc/nginx/conf.d/buffered-log.conf << 'EOF'
server {
    listen 8085;
    server_name localhost;

    root /var/www/static-site;
    index index.html;

    # 配置日志缓冲
    # buffer=32k 缓冲区大小
```

```
# flush=5s      最长刷新间隔
# gzip=1        启用压缩（仅对文件路径有效）
access_log /var/log/nginx/sites/buffered_access.log main
buffer=32k flush=5s;

error_log /var/log/nginx/sites/buffered_error.log warn;

location / {
    try_files $uri $uri/ =404;
}
}
EOF
```

步骤2：测试并重载

```
sudo nginx -t
sudo systemctl reload nginx
```

步骤3：开放端口

```
sudo firewall-cmd --permanent --add-port=8085/tcp
sudo firewall-cmd --reload
```

步骤4：测试缓冲效果

快速发送请求

```
for i in {1..50}; do
    curl -s http://localhost:8085/ > /dev/null
done
```

立即查看日志（可能还没写入）

```
echo "立即查看："
sudo wc -l /var/log/nginx/sites/buffered_access.log
```

等待 5 秒后再查看（缓冲刷新后）

```
sleep 6
echo "5秒后查看："
sudo wc -l /var/log/nginx/sites/buffered_access.log
```

3.7 实验六：使用 logrotate 配置日志切割

操作步骤：

步骤1: 查看系统自带的 Nginx 日志切割配置

```
cat /etc/logrotate.d/nginx
```

步骤2: 创建自定义的日志切割配置

```
sudo tee /etc/logrotate.d/nginx-custom << 'EOF'
```

Nginx 站点日志切割配置

```
/var/log/nginx/sites/*.log {
```

```
    # 每天切割
```

```
    daily
```

```
    # 保留 30 天的日志
```

```
    rotate 30
```

```
    # 压缩旧日志
```

```
    compress
```

```
    # 延迟压缩（保留最近一个不压缩）
```

```
    delaycompress
```

```
    # 日志文件不存在不报错
```

```
    missingok
```

```
    # 日志为空不切割
```

```
    notifempty
```

```
    # 创建新日志文件的权限
```

```
    create 0640 nginx nginx
```

```
    # 使用日期作为后缀
```

```
    dateext
```

```
    dateformat -%Y%m%d
```

```
    # 切割后执行的脚本
```

```
    sharedscripts
```

```
    postrotate
```

```
        # 发送 USR1 信号让 Nginx 重新打开日志文件
```

```
        /bin/kill -USR1 `cat /run/nginx.pid 2>/dev/null` 2>/dev/null
```

```
|| true
```

```
    endscript
```

```
}
```

```
EOF
```

```
# 步骤3: 测试日志切割配置 (不实际执行)
sudo logrotate -d /etc/logrotate.d/nginx-custom

# 步骤4: 手动执行日志切割
sudo logrotate -f /etc/logrotate.d/nginx-custom

# 步骤5: 查看切割结果
ls -la /var/log/nginx/sites/

# 步骤6: 验证 Nginx 正常写入新日志
curl http://localhost:8082/
sudo tail -1 /var/log/nginx/sites/static-site_access.log
```

logrotate 配置说明:

daily/weekly/monthly	切割周期
rotate N	保留 N 个旧文件
compress	压缩旧日志
delaycompress	延迟一个周期再压缩
missingok	文件不存在不报错
notifempty	空文件不切割
create	创建新文件及权限
dateext	使用日期作为文件后缀
sharedscripts	多个文件匹配时只执行一次脚本
postrotate/endscript	切割后执行的命令

3.8 实验七：手动日志切割脚本

有时需要更灵活的日志切割方案，可以使用自定义脚本。

操作步骤:

```
# 步骤1: 创建日志切割脚本
sudo tee /usr/local/bin/nginx-log-rotate.sh << 'EOF'
#!/bin/bash
# Nginx 日志切割脚本
# 用法: nginx-log-rotate.sh [日志目录] [保留天数]
```

```
LOG_DIR="${1:-/var/log/nginx}"
KEEP_DAYS="${2:-30}"
DATE=$(date +%Y%m%d-%H%M%S)

# 切割访问日志
for log_file in "$LOG_DIR"/*_access.log; do
    if [ -f "$log_file" ]; then
        mv "$log_file" "${log_file}.${DATE}"
        echo "切割: $log_file -> ${log_file}.${DATE}"
    fi
done

# 切割错误日志
for log_file in "$LOG_DIR"/*_error.log; do
    if [ -f "$log_file" ]; then
        mv "$log_file" "${log_file}.${DATE}"
        echo "切割: $log_file -> ${log_file}.${DATE}"
    fi
done

# 通知 Nginx 重新打开日志文件
if [ -f /run/nginx.pid ]; then
    kill -USR1 $(cat /run/nginx.pid)
    echo "已通知 Nginx 重新打开日志文件"
fi

# 压缩旧日志 (1天前的)
find "$LOG_DIR" -name "*.log.*" -mtime +0 -not -name "*.gz" -exec
gzip {} \;
echo "已压缩1天前的日志"

# 删除过期日志
find "$LOG_DIR" -name "*.log.*.gz" -mtime +$KEEP_DAYS -delete
echo "已删除${KEEP_DAYS}天前的日志"

echo "日志切割完成: $(date)"
EOF

# 步骤2: 添加执行权限
sudo chmod +x /usr/local/bin/nginx-log-rotate.sh

# 步骤3: 测试脚本
```

```
sudo /usr/local/bin/nginx-log-rotate.sh /var/log/nginx/sites 7
```

步骤4: 查看结果

```
ls -la /var/log/nginx/sites/
```

步骤5: 设置定时任务 (每天凌晨 2 点执行)

```
sudo tee /etc/cron.d/nginx-log-rotate << 'EOF'
```

每天凌晨 2 点切割 Nginx 日志

```
0 2 * * * root /usr/local/bin/nginx-log-rotate.sh  
/var/log/nginx/sites 30 >> /var/log/nginx-rotate.log 2>&1  
EOF
```

步骤6: 验证 cron 配置

```
sudo cat /etc/cron.d/nginx-log-rotate
```

3.9 实验八：日志分析命令实战

操作步骤:

首先产生一些测试数据

```
for i in {1..100}; do  
    curl -s -A "Mozilla/5.0 (Windows NT 10.0)" http://localhost:8082/  
> /dev/null  
    curl -s -A "Mozilla/5.0 (iPhone)"  
http://localhost:8082/css/style.css > /dev/null  
    curl -s -A "Googlebot/2.1" http://localhost:8082/js/main.js >  
/dev/null  
    curl -s http://localhost:8082/nonexistent > /dev/null  
done
```

设置日志文件变量

```
LOG_FILE="/var/log/nginx/sites/static-site_access.log"
```

===== 基础统计 =====

1. 统计总请求数

```
echo "=== 总请求数 ==="
```

```
sudo wc -l $LOG_FILE
```

2. 统计独立 IP 数

```
echo ""
echo "=== 独立 IP 数 ==="
sudo awk '{print $1}' $LOG_FILE | sort -u | wc -l

# 3. 统计每个 IP 的请求数 (前10名)
echo ""
echo "=== IP 请求排行 (前10) ==="
sudo awk '{print $1}' $LOG_FILE | sort | uniq -c | sort -rn | head
-10

# ===== 状态码分析 =====

# 4. 统计各状态码数量
echo ""
echo "=== 状态码分布 ==="
sudo awk '{print $9}' $LOG_FILE | sort | uniq -c | sort -rn

# 5. 查找所有 404 错误
echo ""
echo "=== 404 错误请求 ==="
sudo awk '$9==404 {print $7}' $LOG_FILE | sort | uniq -c | sort -rn |
head -10

# 6. 查找所有 5xx 错误
echo ""
echo "=== 5xx 错误 ==="
sudo awk '$9>=500 && $9<600 {print}' $LOG_FILE | tail -5

# ===== URL 分析 =====

# 7. 统计最热门的 URL (前10)
echo ""
echo "=== 热门 URL (前10) ==="
sudo awk '{print $7}' $LOG_FILE | sort | uniq -c | sort -rn | head
-10

# 8. 统计最大流量的 URL
echo ""
echo "=== 流量最大的 URL ==="
sudo awk '{sum[$7]+=$10} END {for (url in sum) print sum[url], url}'
$LOG_FILE | sort -rn | head -10
```



```
# ===== 时间分析 =====

# 9. 统计每小时请求数
echo ""
echo "=== 每小时请求分布 ==="
sudo awk -F'[::]' '{print $4}' $LOG_FILE | sort | uniq -c

# 10. 统计每分钟请求数（查看高峰）
echo ""
echo "=== 最近请求高峰（按分钟） ==="
sudo awk '{print substr($4,2,17)}' $LOG_FILE | sort | uniq -c | sort
-rn | head -5

# ===== User-Agent 分析 =====

# 11. 统计浏览器/爬虫分布
echo ""
echo "=== User-Agent 分布 ==="
sudo awk -F'"' '{print $6}' $LOG_FILE | sort | uniq -c | sort -rn |
head -10

# 12. 识别爬虫请求
echo ""
echo "=== 爬虫请求数 ==="
sudo grep -i "bot\|spider\|crawler" $LOG_FILE | wc -l

# ===== 来源分析 =====

# 13. 统计来源页面
echo ""
echo "=== 来源页面 TOP 5 ==="
sudo awk -F'"' '{print $4}' $LOG_FILE | grep -v "^-$" | sort | uniq -
c | sort -rn | head -5
```

3.10 实验九：创建日志分析脚本

操作步骤：

```
# 步骤1：创建综合日志分析脚本
sudo tee /usr/local/bin/nginx-log-analyze.sh << 'EOF'
```

```
#!/bin/bash
# Nginx 日志分析脚本
# 用法: nginx-log-analyze.sh <日志文件>

LOG_FILE="${1:-/var/log/nginx/access.log}"

if [ ! -f "$LOG_FILE" ]; then
    echo "错误: 日志文件不存在: $LOG_FILE"
    exit 1
fi

echo "===== "
echo "    Nginx 日志分析报告"
echo "    日志文件: $LOG_FILE"
echo "    分析时间: $(date)"
echo "===== "

echo ""
echo " 【基础统计】 "
echo "-----"
echo "总请求数: $(wc -l < "$LOG_FILE")"
echo "独立 IP 数: $(awk '{print $1}' "$LOG_FILE" | sort -u | wc -l)"
echo "总传输量: $(awk '{sum+=$10} END {printf "%.2f MB\n", sum/1024/1024}' "$LOG_FILE")"

echo ""
echo " 【状态码分布】 "
echo "-----"
awk '{print $9}' "$LOG_FILE" | sort | uniq -c | sort -rn | while read count code; do
    printf "    %s: %s 次\n" "$code" "$count"
done

echo ""
echo " 【IP 请求排行 TOP 10】 "
echo "-----"
awk '{print $1}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10 | while read count ip; do
    printf "    %s: %s 次\n" "$ip" "$count"
done

echo ""
```

```

echo " 【热门页面 TOP 10】 "
echo "-----"
awk '{print $7}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10 |
while read count url; do
    printf "    %s 次: %s\n" "$count" "$url"
done

echo ""
echo " 【404 错误 TOP 5】 "
echo "-----"
awk '$9==404 {print $7}' "$LOG_FILE" | sort | uniq -c | sort -rn |
head -5 | while read count url; do
    printf "    %s 次: %s\n" "$count" "$url"
done

echo ""
echo " 【爬虫访问统计】 "
echo "-----"
bot_count=$(grep -i "bot\|spider\|crawler" "$LOG_FILE" | wc -l)
total_count=$(wc -l < "$LOG_FILE")
bot_percent=$(echo "scale=2; $bot_count * 100 / $total_count" | bc)
echo "    爬虫请求: $bot_count 次 ($bot_percent%)"

echo ""
echo "===== "
echo "    分析完成"
echo "===== "
EOF

# 步骤2: 添加执行权限
sudo chmod +x /usr/local/bin/nginx-log-analyze.sh

# 步骤3: 运行分析脚本
sudo /usr/local/bin/nginx-log-analyze.sh /var/log/nginx/sites/static-
site_access.log

```

3.11 实验十：实时日志监控

操作步骤：

步骤1: 创建实时监控脚本

```
sudo tee /usr/local/bin/nginx-log-monitor.sh << 'EOF'
```

```
#!/bin/bash
```

Nginx 实时日志监控脚本

```
LOG_FILE="${1:-/var/log/nginx/access.log}"
```

```
echo "开始监控: $LOG_FILE"
```

```
echo "按 Ctrl+C 停止"
```

```
echo "-----"
```

使用 tail -f 配合 awk 实时处理

```
tail -f "$LOG_FILE" | awk '{
```

```
    # 提取关键信息
```

```
    ip = $1
```

```
    time = $4
```

```
    request = $7
```

```
    status = $9
```

```
    size = $10
```

```
    # 根据状态码着色
```

```
    if (status >= 500) {
```

```
        color = "\033[31m" # 红色
```

```
    } else if (status >= 400) {
```

```
        color = "\033[33m" # 黄色
```

```
    } else if (status >= 300) {
```

```
        color = "\033[36m" # 青色
```

```
    } else {
```

```
        color = "\033[32m" # 绿色
```

```
    }
```

```
    reset = "\033[0m"
```

```
    # 格式化输出
```

```
    printf "%s%s%s %s %s %s bytes\n", color, status, reset, ip,  
request, size
```

```
}'
```

```
EOF
```

步骤2: 添加执行权限

```
sudo chmod +x /usr/local/bin/nginx-log-monitor.sh
```

步骤3: 创建错误日志监控脚本

```

sudo tee /usr/local/bin/nginx-error-monitor.sh << 'EOF'
#!/bin/bash
# Nginx 错误日志监控脚本

LOG_FILE="${1:-/var/log/nginx/error.log}"

echo "监控错误日志: $LOG_FILE"
echo "按 Ctrl+C 停止"
echo "-----"

tail -f "$LOG_FILE" | while read line; do
    # 根据错误级别着色
    if echo "$line" | grep -q "\[emerg\]\|\[alert\]\|\[crit\]"; then
        echo -e "\033[31m$line\033[0m" # 红色
    elif echo "$line" | grep -q "\[error\]"; then
        echo -e "\033[33m$line\033[0m" # 黄色
    elif echo "$line" | grep -q "\[warn\]"; then
        echo -e "\033[36m$line\033[0m" # 青色
    else
        echo "$line"
    fi
done
EOF

# 步骤4: 添加执行权限
sudo chmod +x /usr/local/bin/nginx-error-monitor.sh

# 步骤5: 测试实时监控 (在一个终端运行)
# sudo /usr/local/bin/nginx-log-monitor.sh
/var/log/nginx/sites/static-site_access.log

# 步骤6: 在另一个终端产生请求测试
# curl http://localhost:8082/
# curl http://localhost:8082/nonexistent

```

3.12 实验十一：配置错误日志级别

操作步骤：

```
# 步骤1: 测试不同错误级别
```

查看当前错误日志级别

```
grep "error_log" /etc/nginx/nginx.conf
```

步骤2: 创建测试配置 (debug 级别)

```
sudo tee /etc/nginx/conf.d/debug-test.conf << 'EOF'
```

```
server {  
    listen 8086;  
    server_name localhost;  
  
    root /var/www/static-site;  
    index index.html;  
  
    # 开启 debug 级别日志 (仅测试用)  
    error_log /var/log/nginx/sites/debug_error.log debug;  
    access_log /var/log/nginx/sites/debug_access.log main;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}  
EOF
```

注意: debug 级别需要 Nginx 编译时启用 --with-debug

检查是否支持

```
nginx -V 2>&1 | grep -o -- '--with-debug' && echo "支持 debug" || echo  
"不支持 debug"
```

步骤3: 测试并重载

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

步骤4: 开放端口

```
sudo firewall-cmd --permanent --add-port=8086/tcp
```

```
sudo firewall-cmd --reload
```

步骤5: 产生请求

```
curl http://localhost:8086/
```

步骤6: 查看详细日志

```
sudo tail -50 /var/log/nginx/sites/debug_error.log
```

步骤7: 测试完成后, 删除或修改 debug 配置 (生产环境不要用 debug)

```
sudo rm /etc/nginx/conf.d/debug-test.conf
sudo systemctl reload nginx
```

四、本章小结

通过本章学习，我们掌握了以下知识点：

理论知识：

1. 访问日志和错误日志的作用与区别
2. 日志格式各字段的含义
3. 错误日志级别及使用场景
4. 日志切割的必要性和常用策略
5. 日志分析的价值和常见维度
6. 日志安全与合规要求

实践技能：

1. 查看和理解默认日志配置
 2. 自定义多种日志格式（详细、JSON、简洁）
 3. 为不同站点配置独立日志
 4. 配置日志缓冲提高性能
 5. 使用 logrotate 配置日志切割
 6. 编写自定义日志切割脚本
 7. 使用命令行工具分析日志
 8. 创建日志分析和监控脚本
 9. 配置不同的错误日志级别
-

五、课后练习

1. 创建一个自定义日志格式，包含请求处理时间、响应大小、状态码，使用 JSON 格式输出

2. 编写一个脚本，统计过去一小时内访问量最大的 10 个 IP 地址
3. 配置 logrotate，每周切割一次日志，保留 12 周，压缩保存
4. 思考题：如果网站使用了 CDN，日志中记录的客户端 IP 会是什么？如何获取真实的用户 IP？